

Analyse et Conception Orientées Objet avec UML

Unified Modeling Language (UML) Généralités

Aminata ZERBO/SABANE

Unified Modeling Language (UML) Généralités



La Modélisation

La Modélisation

« Depuis des temps très anciens, les hommes ont cherché un langage à la fois universel et synthétique, et leurs recherches les ont amenés à découvrir des images, des symboles qui expriment, en les réduisant à l'essentiel, les réalités les plus riches et les plus complexes. Les images, les symboles parlent, ils ont un langage. »

O.M. Aïvanhov

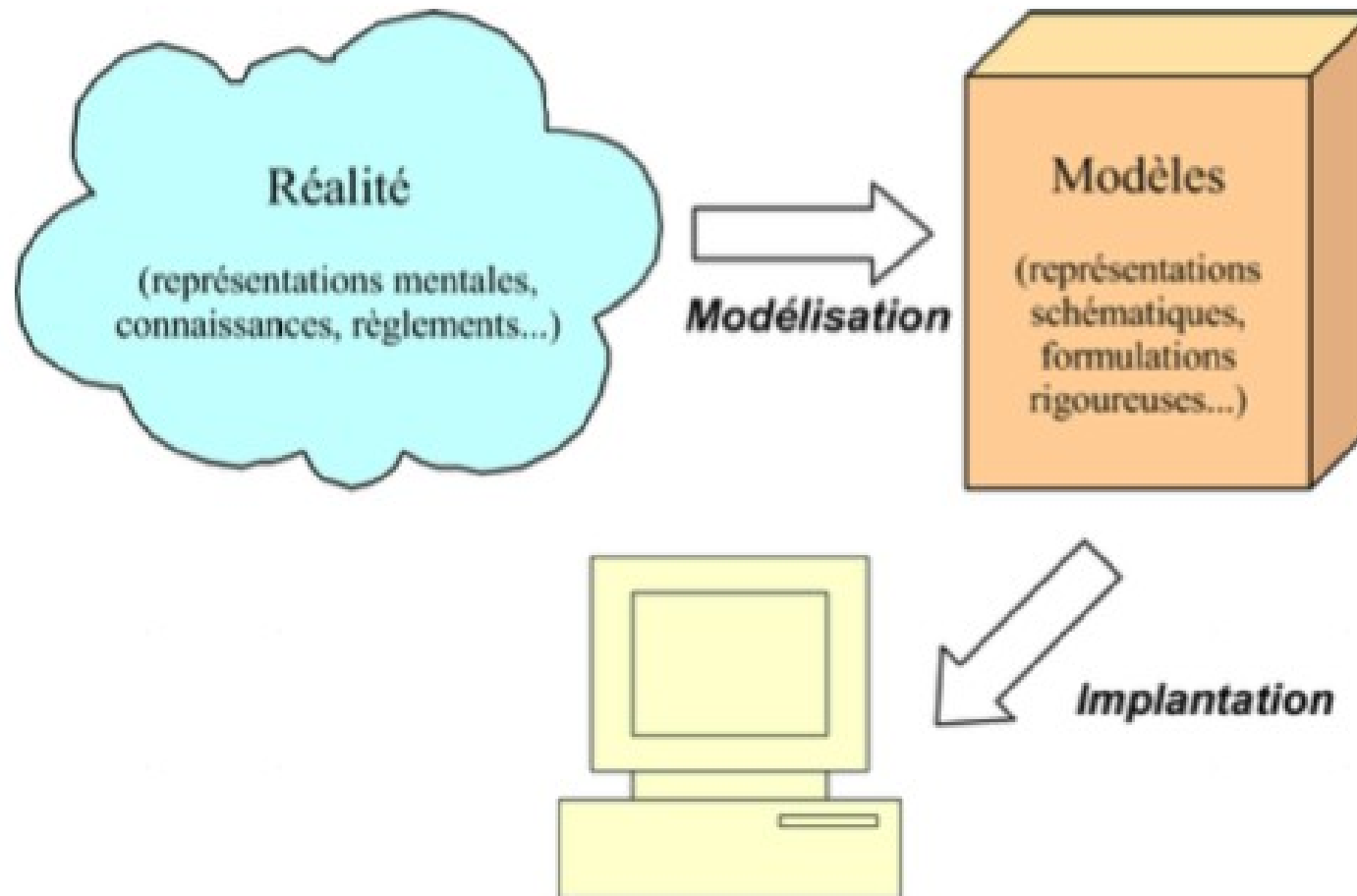
La Modélisation

En informatique, la modélisation consiste à décrire un problème (phase d'analyse) puis à décrire la solution de ce problème (phase de conception) en créant des modèles

L'objectif principal de la modélisation est de maîtriser la complexité pour une meilleure compréhension du système

Modéliser = abstraire la réalité pour mieux comprendre le système à réaliser / réalisé

La Modélisation



Modèle

Un modèle est une abstraction de la réalité

- C'est une représentation abstraite d'un système, d'un phénomène, d'un processus...
- Il permet de réduire la complexité en omettant certains détails peu pertinents pour le contexte
- Un modèle n'est pas "la réalité", mais une vue subjective de la réalité qui reflète des aspects jugés importants

Modèle

Un modèle réduit (décompose) la réalité, dans le but de disposer d'éléments de travail exploitables par des moyens mathématiques ou informatiques :

- modèle / réalité $\simeq$ digital / analogique

Exemples :

Plan électrique d'une maison

Modèle économique

Modèle démographique

Modèle

Objectifs

- Visualiser le système
- Spécifier la structure et le comportement
- Servir de plan pour la construction effective
- Documenter les choix

Modèle

Un modèle permet de :

- faciliter la compréhension du système étudié
Un modèle réduit la complexité du système étudié
- de simuler le système étudié
Un modèle représente le système étudié et permet de reproduire ses comportements.

Présentation d'UML

Unified Modeling Language (UML)

Unified Modeling Language (UML ou en français langage de modélisation unifié) est un langage de modélisation graphique à base de diagrammes

Utilisé en **analyse** et **conception** de systèmes orientés objet, UML permet d'exprimer et d'élaborer des **modèles objet**, indépendamment de tout langage de programmation

UML est la notation de modélisation la plus répandue dans le monde

Unified Modeling Language (UML)

UML est un langage pour :

Visualiser

Chaque symbole graphique de la notation UML possède une sémantique

Spécifier

La notation permet une description précise, complète et sans ambiguïté

Unified Modeling Language (UML)

UML est un langage pour :

Construire

Une partie du code des classes peut être générée automatiquement à partir des diagrammes UML

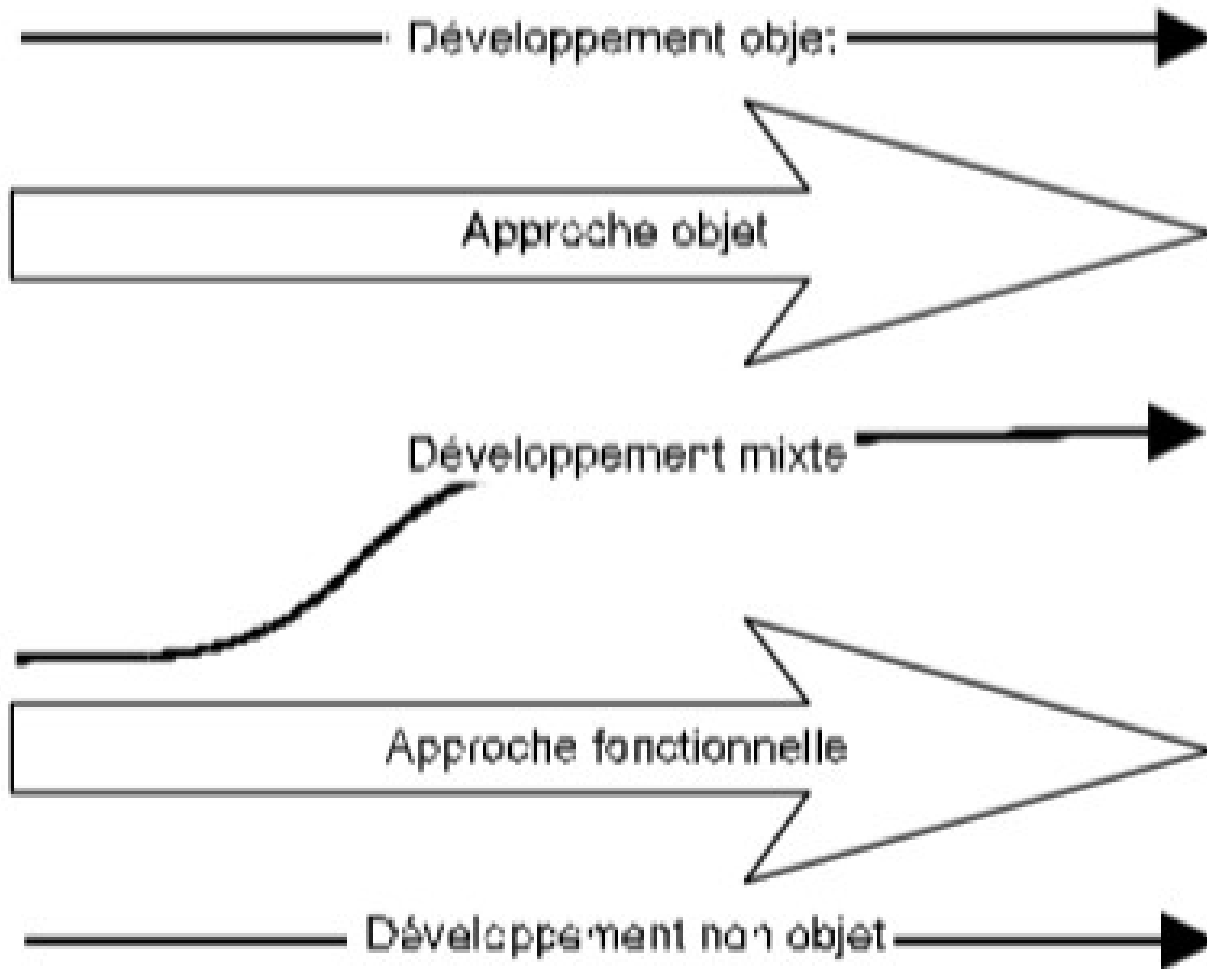
La rétrospection à partir du code est également possible

Documenter

Les différents diagrammes, notes, contraintes, exigences constituent une documentation du système qui facilite les échanges entre les différents intervenants

Origine d'UML

Origine d'UML



Milieu des années 80 :
Les avantages de l'approche orientée objet sont reconnus

Problème : Pas de méthode d'analyse orientée objet

Nécessité d'une méthode spécifiquement objet pour tout le cycle de développement

Origine d'UML

Plus de 50 méthodes objet apparaissent pour répondre à cette nécessité

Booch, Classe-Relation, Fusion, HOOD, OMT, OOA, OOD, OOM, OOSE...

Cette prolifération de méthodes crée de la confusion

Pas de consensus sur ce qu'est l'approche objet

Aucune méthode n'émerge réellement mais certaines sont dominantes

Origine d'UML

UML nait de la fusion des trois plus célèbres méthodes objet du milieu des années 90 :

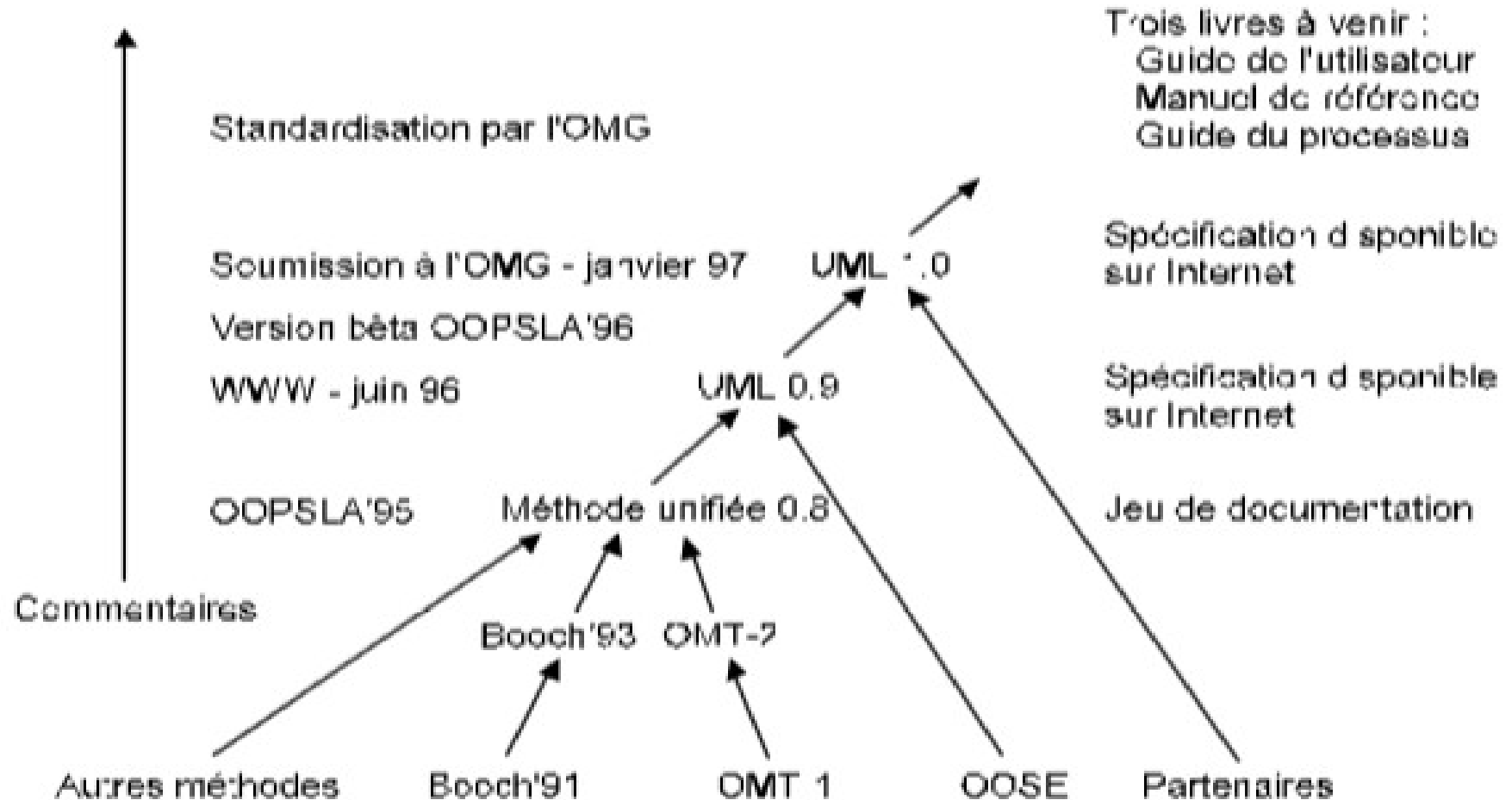
- **Booch** de Grady Booch
- **OMT** (*Object Modeling Technique*) de James Rumbaugh
- **OOSE** (*Object-Oriented Software Engineering*) de **Ivar Jacobson**

Origine d'UML

Los Tres Amigos Booch, Jacobson et Rumbaugh fixent quatre objectifs pour la méthode unifiée :

1. Représenter des systèmes entiers par des concepts objets
2. Etablir un couplage explicite entre les concepts et les artefacts exécutables
3. Prendre en compte les facteurs d'échelle inhérents aux systèmes complexes et critiques
4. Créer un langage de modélisation utilisable à la fois par les humains et les machines.

Origine d'UML



Origine d'UML

Origine	Élément
Booch	Catégories et sous-systèmes
Embley	Classes singletons et objets composites
Fusion	Description des opérations, numérotation des messages
Gamma et al.	Frameworks, patterns et notes
Harel	Automates (<i>Statecharts</i>)
Jacobson	Cas d'utilisation (<i>use cases</i>)
Meyer	Pré- et post-conditions
Odell	Classification dynamique, éclairage sur les événements
OMT	Associations
Shlaer-Mellor	Cycle de vie des objets
Wirfs-Brock	Responsabilités (<i>CRC</i>)

**Origine des
Principaux
Concepts
d'UML**

Caractéristiques d'UML

Caractéristiques d'UML

UML n'est pas une méthode objet

- C'est un langage de modélisation objet

Une méthode définit une démarche reproductible pour obtenir des résultats fiables

Elle propose un processus qui régit l'enchaînement des activités à réaliser pour atteindre les résultats

UML est indépendant de la méthode mise en œuvre

UML a été adopté par la plupart des méthodes orientées objet (2TUP, RUP)

Caractéristiques d'UML

UML est un standard (une norme)

Il est dans le domaine public et accessible par tous

UML est un langage pseudo-formel fondé sur un métamodèle

UML normalise les concepts OO

Un métamodèle définit avec précision:

- les éléments de modélisation (les concepts manipulés par le langage),
- la sémantique de ces éléments (leur définition et le sens de leur utilisation).

Un métamodèle limite les ambiguïtés et encourage la construction d'outils

Caractéristiques d'UML

UML est un support de communication

UML permet de visualiser une solution objet à l'aide de sa notation graphique

Cette notation limite les ambiguïtés et les incompréhensions

UML est un langage universel

Sa notation est indépendante de tout :

- langage d'implémentation
- domaine d'application
- processus de développement

Diagrammes UML

Diagrammes UML

UML permet de modéliser un système à l'aide de diagrammes

Un diagramme UML est une représentation graphique qui représente un aspect précis du système à réaliser

La version UML2 compte Quatorze (14) diagrammes classés en deux groupes

- Les diagrammes structurels (sept diagrammes)
- Les diagrammes comportementaux (sept diagrammes)

La combinaison de ces diagrammes fournit une vue complète des aspects statiques et dynamiques d'un système

Diagrammes UML

Diagrammes Structurels

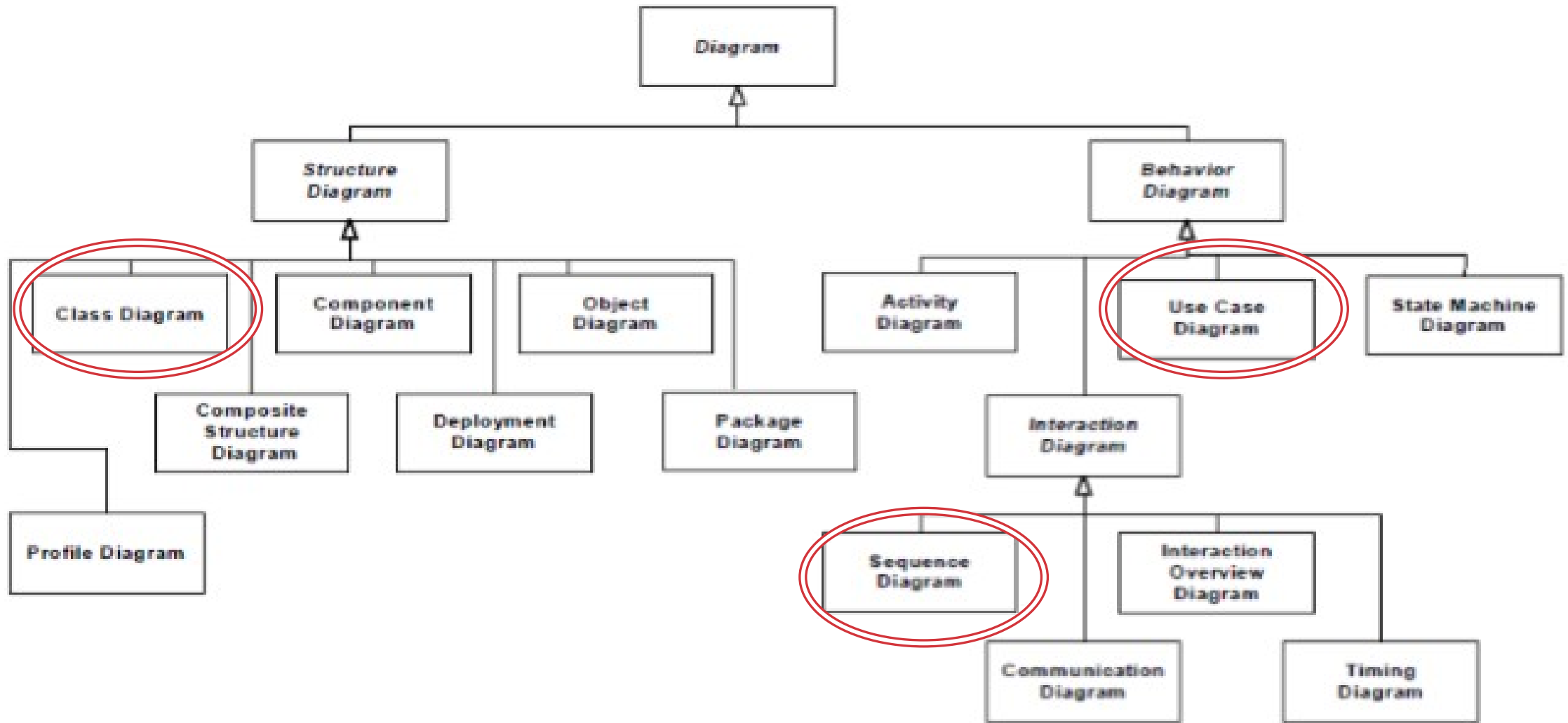
Ces diagrammes définissent l'architecture statique du logiciel

Ils permettent de modéliser les entités du système (classes, objets, interfaces, composants physiques, ...) et les relations entre ces entités

Diagrammes Comportementaux

Les diagrammes comportementaux permettent de décrire le comportement du système dans son environnement, les interactions entre ses entités et ses états durant son exécution

Diagrammes UML



Modéliser avec UML

Modéliser avec UML

UML n'est pas une méthode mais un langage qui permet de représenter des modèles

UML ne définit pas le processus d'élaboration des modèles

Néanmoins, les auteurs d'UML indiquent la démarche à suivre pour réussir son projet

Cette démarche doit être :

- o itérative et incrémentale
- o guidée par les besoins des utilisateurs du système
- o centrée sur l'architecture logicielle.

Modéliser avec UML

Une démarche itérative et incrémentale

C'est une démarche qui construit et valide par étapes

Elle consiste à modéliser le système par affinements successifs (les itérations) afin de mieux l'analyser et le comprendre

Une démarche guidée par les besoins des utilisateurs

La satisfaction des besoins utilisateurs est le principal objectif du logiciel

- Les utilisateurs définissent ce que doit être le système et son périmètre
- Ils doivent être impliqués à toutes les phases du cycle de développement afin que leurs besoins soient pris en compte et pleinement satisfaits

Modéliser avec UML

Une démarche centrée sur l'architecture logicielle

L'architecture décrit des choix stratégiques déterminants pour la qualité du logiciel (adaptabilité, performances, fiabilité...).

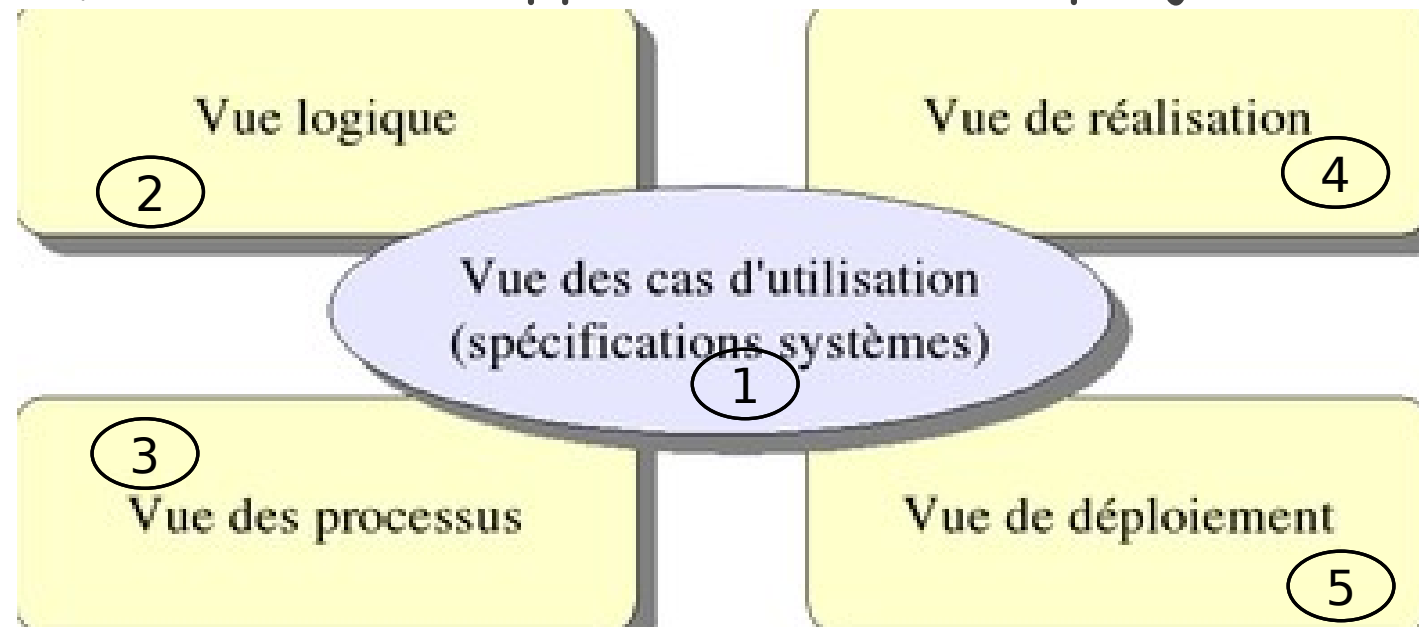
Elle constitue donc une composante clé du développement logiciel

UML a été fortement inspiré par le modèle 4+1 vues de Kruchten qui définit les différentes vue d'un système

4+1 vues de Kruchten

Le modèle 4+1 vues de Kruchten définit cinq perspectives, indépendantes et complémentaires du système.

- Chaque perspective (vue) présente le système selon un point de vue différent (celui du développeur, du chef de projet, des clients, ...).



4+1 vues de Kruchten

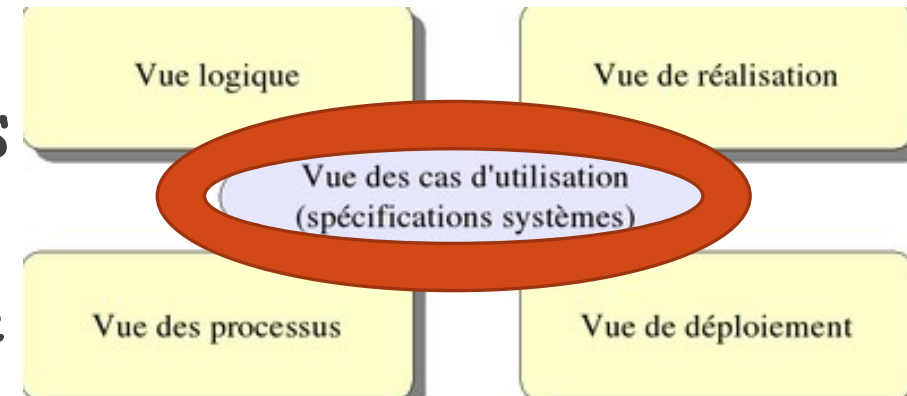
La vue des cas d'utilisation

C'est la vue centrale qui guide toutes les autres vues

Elle définit les besoins des clients du système

Un cas d'utilisation est un ensemble de scénarios d'utilisation

C'est une séquence d'interactions d'un utilisateur (acteurs) avec le système



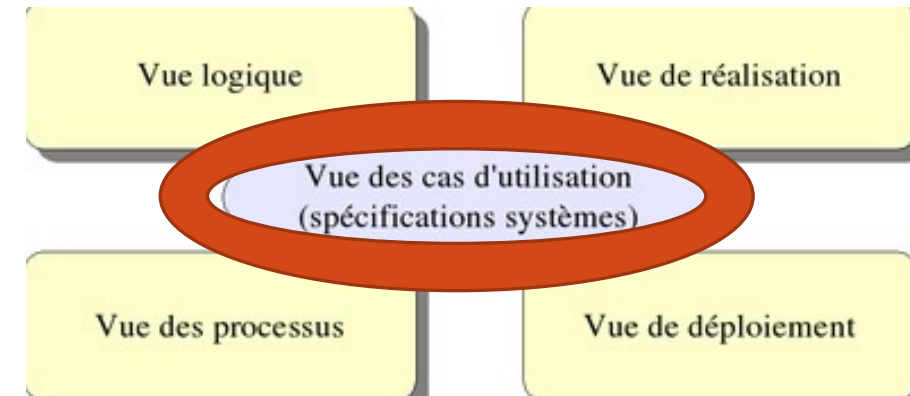
4+1 vues de Kruchten

La vue des cas d'utilisation

Cette vue valide les autres vues et assure la cohérence globale

La vue des cas d'utilisation est construite en premier à partir du cahier des charges

Elle précise les fonctionnalités du système et son périmètre



Principaux diagrammes
élaborés dans cette vue
Diagramme de cas d'utilisation

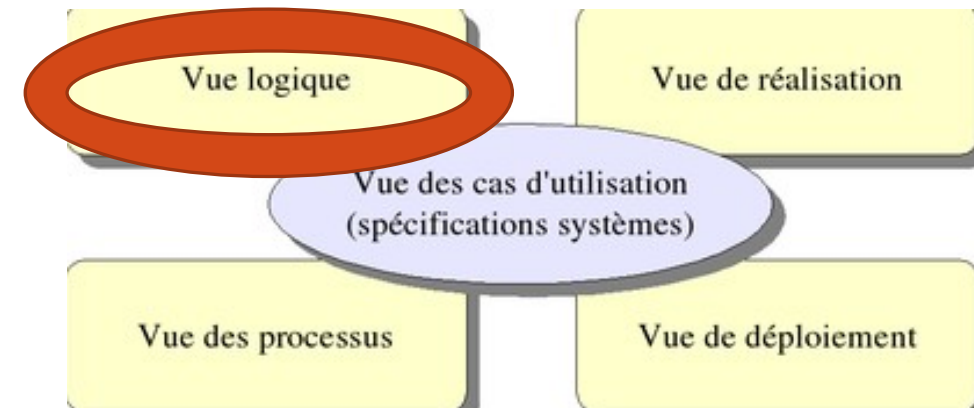
4+1 vues de Kruchten

La vue logique (Abstraction et Encapsulation)

Elle constitue la principale description de l'architecture du système

Cette vue décrit le système en termes d'objets et de classes

Elle identifie les éléments du domaine, ainsi que les relations et interactions entre ces éléments



Principaux diagrammes élaborés dans cette vue
Diagrammes de classes
Diagrammes d'objets

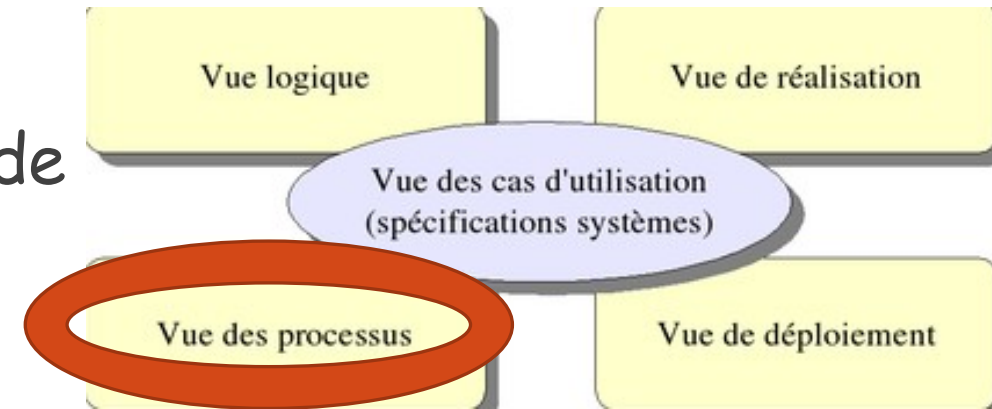
4+1 vues de Kruchten

La vue des processus

Elle permet de vérifier le respect des contraintes de fiabilité, d'efficacité et de performances des systèmes multitâches

Cette vue présente

- La décomposition du système en terme de processus (tâches)
- Les interactions entre les processus (leur communication)
- La synchronisation et la communication des activités parallèles (threads)



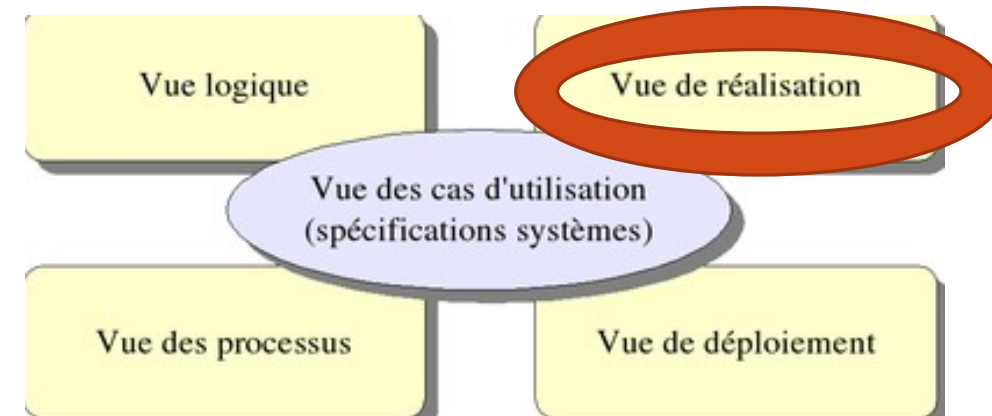
Principaux diagrammes
élaborés dans cette vue
Diagrammes dynamiques
(séquences, activités, états-
transition, communication)

4+1 vues de Kruchten

La vue de réalisation ou de développement

Elle représente l'organisation des composants (exécutable, codes source, paquetages, bibliothèque, etc.) dans l'environnement de développement

Vue des développeurs



Principaux diagrammes
élaborés dans cette vue
Diagramme de composants

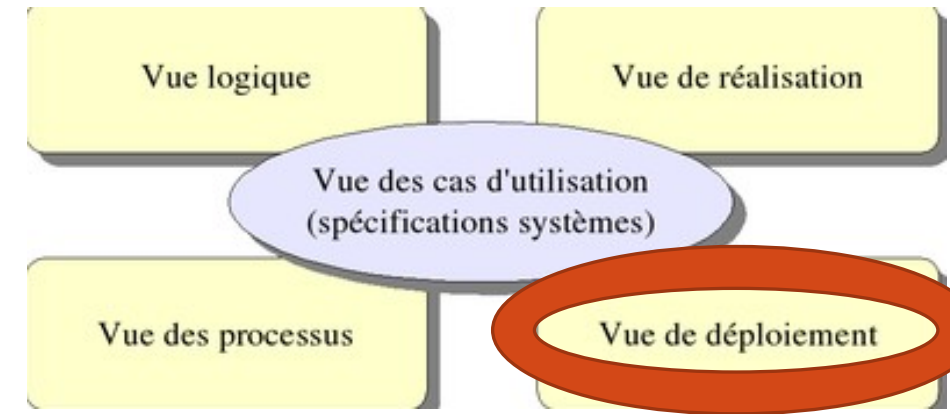
4+1 vues de Kruchten

La vue « physique » ou de déploiement

Elle décrit le système dans son environnement d'exécution

Cette vue présente les différentes ressources matérielles et l'implantation logicielle dans ces ressources

- Disposition et nature physique des matériels
- Implantation des modules principaux sur les nœuds du réseau



Principaux diagrammes
élaborés dans cette vue
Diagramme de déploiement

Références

Modélisation Objet avec UML - Pierre Alain MULLER

COO avec UML & Java - Poly de Denis Conan et Jean-Luc Raffy

www.wikipedia.org

<http://uml.free.fr/>

INF4030 Conception Orientée Objet (COO)

Unified Modeling Language (UML) Généralités

Aminata ZERBO/SABANE